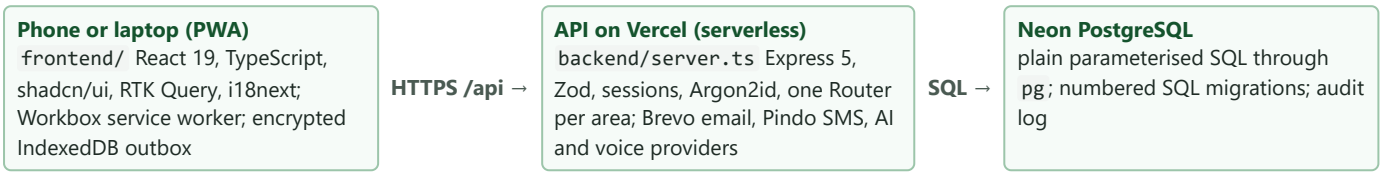
Rinda Health: Technical Report

Final Version of the Product, BSc Software Engineering capstone, African Leadership University · Mugemanyi Manzi David · Supervisor: Emmanuel Annor · November 2026

Live app: rinda-health.vercel.app · Repository: github.com/DLOADIN/Rinda-health

1. Product overview and architecture

Rinda Health is a bilingual (English and Kinyarwanda), offline-first progressive web app (PWA). It helps Rwandan adults aged 18–35 screen six non-communicable disease (NCD) risk factors: tobacco, alcohol, physical activity, fruit and vegetables, body size and blood pressure. People can also take an optional mental-wellness check (PHQ-9, GAD-7). Each area gets its own explained, educational result, with a short weekly plan and links to real services. The product is **educational, not diagnostic**, and it is not an emergency service. Four roles share one app: participant, facilitator, researcher-admin and reviewer.



2. Scope alignment and feature validation

The approved scope is the capstone proposal (18 September 2026): 20 functional requirements (its Table 14), 12 non-functional requirements (Table 15) and the evaluation metrics (Table 13). Each is mapped below to its status and evidence in the repository. Status key: Done · Partial · Not done.

2.1 Functional requirements from the proposal

ID	Requirement (purpose)	Status	Evidence (repository)
FR-01	Show non-diagnostic, non-emergency purpose first	Done	`features/landing/`, consent screen; disclaimer on every result
FR-02	Age eligibility and versioned consent	Done	`consent-page.tsx` (18–35 check); `consent_events` table stores `consent_doc_version`, never updated
FR-03	Pseudonymous access, minimal profile	Changed	Email/phone + password or Google sign-in (see 2.4); staff see only `maskIdentifier()` labels
FR-04	Versioned questionnaire; encrypted draft	Partial	Versioned ruleset JSON; finished check-ups saved AES-GCM encrypted offline (`lib/offline.ts`). Resuming a half-finished form is not built
FR-05	Skip any sensitive question or the wellness module	Done	"Skip" on every sensitive item, stored as a missingness code (`Answer<T>` in `types.ts`), never as 0
FR-06	Immediate pause/exit to a neutral screen	Done	`features/exit/neutral-screen.tsx`; Exit in every participant header; `e2e/shell.spec.ts`
FR-07	Validate biomarkers with units and ranges	Done	`packages/rules/src/validate.ts` and Zod schemas, on phone and server
FR-08	Idempotent submit; tolerate lost connection	Done	`Idempotency-Key` + SHA-256 body hash; encrypted outbox; QR hand-over
FR-09	Deterministic, versioned, per-domain results	Done	`packages/rules/src/score.ts`; 79 golden cases; no combined score
FR-10	Factors, completeness, source, limits, advice	Done	`DomainResult.completeness`; "Why this result"; "How results are calculated" page
FR-11	On-device PHQ-9 item-9 check, safety screen	Done	`checkItem9()` in `safety.ts` runs before any fetch; a component test proves it
FR-12	"Get help now" on every screen, offline too	Done	`components/get-help.tsx` bundled in the app shell; e2e offline test
FR-13	Safety events without response content	Done	`safety_events` table (tier, time, action); queued offline, then sent
FR-14	History marked with ruleset version	Done	`features/results/` history and charts; `ruleset_version` stored on every assessment
FR-15	No answers or results in logs or analytics	Done	pino redaction; no analytics SDK; `scripts/check-logs.mjs` sentinel search in CI

FR-16	English and Kinyarwanda, switch any time	Done*	`packages/content/{en,rw}` with a key parity check. *Bilingual review pending
FR-17	Usability and baseline-comparison feedback	Partial	SUS form (`sus-page.tsx`) and the facilitator task-completion log are built; the non-personalised baseline page is not
FR-18	Withdrawal and deletion requests	Done	`POST /participant/deletion-requests`; admin queue and `/complete` route
FR-19	De-identified research export with versions	Partial	`GET /admin/research-report`: task completion, SUS bands, bands per domain, safety tiers, languages (aggregate only)
FR-20	Serve only approved content (approver, date)	Done	Content workflow with two approvals; approver and date stored; only published versions served

Built beyond the proposal (after ethics approval, behind feature flags): the Ask Rinda AI guide and chat (validated, crisis-guarded, 15 questions a day); the voice check-up; email and SMS result summaries; blood pressure read from a photo; QR hand-over with a facilitator QR-session page; and dashboards for four roles.

2.2 Non-functional requirements from the proposal

Category	Status	Evidence
Security	Done	HTTPS only with Helmet headers and CSP; Argon2id; HttpOnly SameSite sessions; `requireRole()` on staff routes; sign-in rate limits; secrets in Vercel env; `npm audit` in CI
Privacy	Done	Data minimisation, masked staff view, consent switches, deletion workflow, no third-party analytics
Installability, offline	Done	Web manifest and Workbox service worker; the live site reloads with the network off (measured, 4.3)
Compatibility	Partial	Chromium and WebKit at 360 px pass in CI; Samsung Internet (Chromium engine) still to be checked on a real phone
Reliability	Partial	Idempotent retry and an ordered outbox; Neon backups exist, but a restore drill is still to be run
Performance	Done	Cached app opens in 0.13–0.32 s (target < 1 s); first visit 1.3–2.7 s (4.3)
Accessibility	Done	axe-core scans, 52 contrast checks, word + icon bands, 44 px targets, labelled controls
Localization	Done*	All text in resource files in both languages; *Kinyarwanda review and Kinyarwanda voice audio pending
Explainability	Done	Each assessment stores its answers and ruleset version, so its result can be reproduced exactly
Maintainability	Done	Typed Zod contracts, numbered SQL migrations, ESLint/Prettier, golden tests (Section 5)
Observability	Done	`x-request-id` correlation ID on every request; structured pino logs without health data
Research integrity	Partial	Missingness codes and ruleset versions stored; a row-level export with a data dictionary is not built

2.3 Evaluation metrics (proposal Table 13)

Metric and target	Result so far
Scoring reproducibility: 100% identical	**Met: 79 golden cases identical in Node, Chromium and WebKit**
Duplicate prevention: zero duplicates	**Met: same-key replay and 409-mismatch integration tests**
Restricted data logging: zero sensitive values	**Met: the log sentinel search after the e2e run finds none**
Skip, pause and exit control success: 100%	**Met: Exit and Get help e2e tests, online and offline**
Core flow on Chrome, Safari and Samsung Internet	**Chromium and WebKit met; the Samsung Internet device check is pending**
Task completion ≥ 80%, SUS ≥ 68, comprehension ≥ 80%, relevance, usefulness, cultural fit	**To be measured in the November pilot (15 participants, 5 key informants); the SUS form, task log and research report that collect them are built**

2.4 Deviations from the proposal and justification

Change	Reason
Study code sign-in became email/phone + password or Google (FR-03)	Testers and staff needed self-service accounts and password recovery. Email and phone are used only to sign in and for opted-in result messages, and staff never see them unmasked.
Generative AI, which the proposal left out of the MVP, was added	Allowed after ethics approval. It sits behind flags, receives only structured result codes, is validated against approved content, and never sets a result or writes safety or referral text (proposal 1.10.7).
Render → Vercel; Flyway/Alembic → plain SQL migrations; React 18 → 19; Node 20 → 24	Vercel gives free serverless hosting with preview deploys. Alembic needs Python, which the project excludes. Newer LTS versions keep security support.
Draft resume, baseline page and row-level export deferred	Time went first to safety, offline sync and testing. All three are planned before the pilot.

3. Technical functionality

Feature	How it works (technical logic)
Accounts and roles	Routes in the `auth` router. Passwords are hashed with Argon2id; Google sign-in uses OpenID Connect with PKCE. Session cookies are HttpOnly and SameSite, with 8 h idle and 12 h absolute limits. `requireRole()` guards every staff route, and each staff action is written to the audit log.
Check-up and scoring	React Hook Form validates with Zod schemas from `packages/schemas`. A skipped answer is stored as a missingness code, never as 0. `scoreAssessment()` is a pure function: it runs on the phone for instant and offline results, and again on the server, which stores the authoritative copy with its ruleset version.
Safety	`checkItem9()` runs on the phone before any network call and opens the safety screen. `checkChatQuestion()` blocks crisis phrases and personal data before any AI call. In the voice check-up, item 9 can only be answered by tapping, to avoid mis-hearing.
Offline and sync	The service worker precaches the app shell. Offline check-ups are encrypted with AES-GCM 256 (Web Crypto, a non-exportable device key, `lib/storage/vault.ts`) in IndexedDB, then sent in order when online with an `Idempotency-Key`. The server stores a SHA-256 hash of the canonical body, so a resend returns the first result.
AI guide and chat	A provider chain (Gemini → Groq → OpenRouter) is tried in order. Every answer passes `validateGuideAnswer`, which accepts only approved source IDs and numbers taken from the person's own results; otherwise the approved template is shown. Each person can ask 15 questions a day.
Voice check-up	Each question is spoken by a text-to-speech chain (Groq → ElevenLabs → Gemini → browser voice). Clips are cached in `voice_clips` under a SHA-256 key, and only approved content IDs can be spoken. Answers are heard with the browser's speech recognition and parsed by a deterministic parser (`voice-parse.ts`) into the same form.
Email and SMS	After a check-up is saved, `sendReportEmail` (Brevo) and `sendSmsReport` (Pindo) run after the response through Vercel `waitUntil`. Each respects the person's on/off switch. Outcomes are logged to `message_events` as status only, with no content.

Frontend–backend–data flow. RTK Query (`lib/api.ts`) calls `/api/v1/…`. Express parses each body with a strict Zod schema (unknown fields → 400 with a stable error code). Plain parameterised SQL (`$1`) writes to Neon, and schema changes are numbered SQL migrations. RTK Query cache tags refresh the affected screens. Logs (pino) never contain answers, results, passwords or tokens, and a sentinel test enforces this.

4. Testing, validation and performance

4.1 Testing strategies

Strategy	Why chosen	How it was conducted	Result	Improvement made
Unit + golden tests (Vitest)	Scoring must be exact and identical on every device	79 scoring and 7 planner golden fixtures plus rule, wellness, guide and chat-guard tests in `packages/rules/test`	**207 passed**	BMI compared as an integer (`bmiTimesTen`) so floating-point rounding can never move a value across a band edge
Integration tests (Supertest + real Postgres)	Routes, database and rules must work together	`backend/server.test.ts`: every test runs in a throwaway Neon schema with mocked Brevo, Pindo and AI providers	**44 passed**	Found that emails never sent on Vercel (fixed with `waitUntil`); fixed consent defaults
Component tests (Testing Library)	UI logic such as safety screens and voice flow	`frontend/src/**/*.test.tsx`, e.g. the item-9 screen opens before any fetch; voice answers fill the form	**69 passed**	Fixed stale answers in the voice flow; item 9 made tap-only after a safety review
End-to-end + compatibility (Playwright)	Real browser behaviour on phones	`e2e/*.spec.ts` on Pixel 7 (Chromium) and iPhone 14 (WebKit): golden parity, Exit and Get help online and offline, privacy	**Pass in CI**	Get help made reachable inside the chat drawer
Accessibility	WCAG 2.2 AA for all users	axe-core scans in Playwright; `check-contrast.mjs` tests 52 colour pairs	**0 violations**	Bands show a word and an icon, never colour alone; 44 px tap targets
Performance budget	Low-end phones on slow data	`check-size.mjs` fails the build above 250 KB gzip first-load JS	**249.8 KB**	Lazy-loaded chat, charts, staff pages, OCR and translation bundles
Live service tests	Real delivery cannot be mocked away	`npm run sms:check`, Brevo send, `models:check`; voice round-trip (TTS → speech-to-text → parser)	**All delivered; voice 10/10**	Corrected provider variable names; reordered voice providers by speed

Continuous integration (`.github/workflows/ci.yml`) runs lint, typecheck, all tests, content validation, build, size and contrast checks, Playwright on Chromium and WebKit, a log sentinel search and `npm audit` on every push.

4.2 Testing with different inputs

Input	Expected and observed behaviour
Complete normal check-up	A separate result per area, a plan, and an email/SMS if switched on
Blood pressure skipped	BP shows "Not answered"; the other areas still score (missing ≠ 0)
BP 300/40 or systolic below diastolic	Field validation message; the reading is refused on phone and server
PHQ-9 item 9 answered 1–3	Safety screen opens immediately, before any network call
Chat: crisis phrase / phone number / football question	`CHAT_SAFETY` safety message / `CHAT_PERSONAL` refusal / polite off-topic reply; nothing is sent to the AI for the first two
16th chat question or 6th check-up in a day	HTTP 429 `CHAT_LIMIT` / `DAILY_LIMIT` with a calm message
Same check-up sent twice / same key, different body	Second request returns the first result / HTTP 409 `IDEMPOTENCY_MISMATCH`
Airplane mode during a check-up	"Saved on this phone"; sent automatically when back online, no duplicate
Request body with an extra field; repeated wrong passwords	HTTP 400 validation error that does not echo values; sign-in rate-limited

4.3 Performance and compatibility

The live site was measured with a Playwright script: cold load, median of 3 runs, on 1 November 2026. The low-end profile emulates a mid-range Android on Slow 4G (1.6 Mbps, 150 ms latency) with a 4× CPU slowdown.

Environment (engine, device, network)	First paint	Largest paint	Page loaded	Transferred	JS heap
Chromium (Chrome/Edge), Windows laptop 1366×768, fast	1.49 s	1.82 s	1.34 s	365 KB	9.5 MB
Chromium, Pixel 7 (Android), fast	1.65 s	2.21 s	1.49 s	365–607 KB	9.5 MB
Chromium, Pixel 7, **low-end: Slow 4G + 4× CPU**	3.03 s	3.95 s	2.68 s	365 KB	9.5 MB
WebKit (Safari engine), iPhone 14 (iOS), fast	2.09 s	3.32 s	2.47 s	373 KB	n/a

Analysis. Even on the low-end profile the first content shows in about 3 s, under the 4 s "poor" Largest Contentful Paint line; on fast networks it is 1.8–2.2 s on Chromium, close to the 2.5 s "good" target. The page stays light (about 365 KB transferred, 9.5 MB JavaScript heap) because heavy features load only when opened. On a repeat visit the service worker serves the app from its cache: it loaded in 0.13 s on the fast phone profile and 0.32 s on the low-end one (median of 3), well inside the proposal's 1 s target, and it reloaded fully with the network switched off. Compatibility is covered by running the same end-to-end suite on Chromium (Chrome, Edge and Samsung Internet on Android and Windows) and WebKit (iPhone and iPad Safari) at 360 px phone width and on laptop screens. Remaining limits: speech recognition is not available in every browser (the voice check-up hides itself and the form stays usable), and Kinyarwanda voice is not available yet.

5. Code quality and technical implementation

5.1 Structure and separation of concerns

The code is an npm-workspaces monorepo. `frontend/src/features/*` holds one folder per area (auth, consent, participant, results, guide, profile, dashboard, staff, sync). `components/` holds reusable compositions (AppShell, GetHelp, BandBadge, ConfirmAction), and `lib/` holds infrastructure (API client, offline outbox, encrypted storage, i18n, BP reader, voice). The most important logic, scoring, safety and planning, is not in the UI or the API. It lives in `packages/rules` as pure, framework-free functions, so it has one source of truth, works offline and is easy to test. `packages/schemas` owns every data contract, and `packages/content` owns every participant-facing word. The API is a single deployable file for simple serverless hosting. Inside it, each area is its own Express `Router` (15 routers: auth, account, participant, facilitator, admin, content, dashboard, guide, chat, voice, messaging and others), built by `createApp()` from injected dependencies.

5.2 Object-oriented principles applied

Principle	Where and how
Abstraction	`Ruleset`, `DomainResult`, `AssessmentResult`, `Plan` types (`packages/rules/src/types.ts`). Callers use `scoreAssessment()` without knowing any per-area rule.
Encapsulation	`lib/offline.ts` keeps the outbox queue private and exposes only `addToOutbox`, `flushOutbox` and `useOutbox`; `lib/storage/vault.ts` hides the non-exportable AES key behind encrypt/decrypt calls.
Inheritance	`AppError extends Error` carries an HTTP status and a stable code, so one error handler serves every route. `GuideBoundary extends React.Component` isolates guide failures.

Polymorphism	AI and voice providers share one interface: the chain calls `askGemini`, `askOpenAiStyle` (Groq, OpenRouter, Cerebras, Mistral) or `speakGroq` / `speakElevenLabs` / `speakGemini` the same way and falls back on failure. The seven per-area scorers share one signature in a `SCORERS` strategy map, so `scoreDomain()` treats them alike.
Dependency injection	`createApp({ config, logger, db, fetchFn })`, so tests inject a fake `fetch` and a throwaway database schema with no code changes.
Reuse and single responsibility	Shared packages are imported by both web and API. Schemas only validate, rules only compute, content only holds text. shadcn/ui components are themed once in `rinda-theme.css`.

5.3 Algorithms and problem solving

Problem	Solution
Identical results on every device	Pure, versioned rules with integer arithmetic (BMI × 10, ages and minutes as integers). 79 golden cases run in Node, Chromium and WebKit to prove client–server parity.
Reading a BP monitor from a photo	A pipeline that runs in the browser: grayscale, Otsu thresholding, morphological filtering with the van Herk/Gil-Werman sliding min/max (O(n) per row, whatever the window size), line and speckle removal, projection profiles to find digit rows, shear estimation for slanted digits, rotation for tilted photos, then seven-segment decoding. Several frames must agree before a value is accepted.
Choosing next steps	A priority planner: urgent areas first, then a fixed order, at most 3 focus areas. Targets are rounded to practical steps, e.g. "+20 minutes on 3 days".
Moving data with no internet (QR)	The phone seals the check-up for the server only: ECDH P-256 key agreement, then HKDF, then AES-GCM 256, compressed to fit in a QR code. The facilitator relays it without being able to read it.
No duplicates on bad networks	Idempotency key plus a SHA-256 hash of the canonical, key-sorted JSON body. Same key with the same body → replay; with a different body → 409.
Safe AI output	A validator whitelists source IDs and checks every number against the person's own results. The provider chain falls back on errors and time-outs, and a crisis and personal-data guard runs first.
Understanding spoken answers	A deterministic parser normalises text, maps number words to digits, matches yes/no and choice synonyms, detects commands (skip, repeat, stop) and flags off-topic speech. A crisis check runs on every utterance.
Cheap, reliable SMS	Typographic characters (curly quotes, dashes, ellipses) are replaced so the text stays in the GSM-7 alphabet and each part is billed at the lowest rate. An optional allow-list (`SMS_ONLY_TO`) limits sends during testing.

5.4 Quality practices, limitations and future work

- TypeScript strict mode with no `any`; ESLint and Prettier enforced; kebab-case files and named exports; Zod at every boundary with stable error codes; no hard-coded participant text (all via i18n, both languages).
- Privacy by design: data minimisation, encrypted local storage, masked identifiers for staff, audit log, consent switches, and no third-party analytics.
- Over 200 commits with short conventional messages, plus CI on every push.
- **Limitations and next steps:** split the API file's routers into separate modules after the pilot; build the What-if screen and clinic card; record and review Kinyarwanda audio; complete native-speaker and clinician content review; run the November pilot (15 participants, 5 key informants) with SUS usability scoring.